

## TECHNICAL ASSESSMENT

# Full-Stack Developer

Choose Assignment A or Assignment B

---

## Overview

This assessment has been designed to evaluate your frontend **and** backend capabilities as a full-stack engineer. You are presented with **two assignment options**. Pick the **one** that best lets you show how you build, reason, and ship — you only need to complete a single assignment.

- **Assignment A — Dynamic Form Builder Engine.** A configuration-driven engine that defines, validates, and stores dynamic forms, with a UI to render and submit them.
- **Assignment B — Submission & Approval Workflow.** A two-sided app built around a correctly enforced status workflow with an audit trail.

We care more about **clean, well-reasoned, well-tested code on a small surface** than about breadth of features. If you run out of time, ship less but ship it solid, and note what you cut.

## Use of AI Tools

AI assistants (e.g. coding copilots, chat-based assistants) are **allowed and encouraged**. In fact, these assignments themselves were drafted with the help of AI — so we have no objection to you using the same kinds of tools in your work.

What we do ask is transparency. In your README, briefly document:

- **Which tools** you used (name them).
- **How** you used them — e.g. scaffolding, generating tests, debugging, writing docs, or reviewing your own code.
- **What you verified yourself.** You should understand and be able to explain every part of what you submit, including anything AI-generated.

## Submission Requirements

1. **Git repository.** Submit your work as a Git repository — public, or private with access granted to us.
2. **Hosted / deployed.** The project must be hosted so we can use it without setting it up locally. Include the live URL (and any test credentials) in your README.
3. **README in the submission.** The submission folder must contain a top-level README.md.

Your README should cover, at minimum:

- How to run the project locally (the documented steps must bring up the database and backend, and start the frontend).

- Your data model and the key design decisions behind it.
- The trade-offs you made and what you would add or change with more time.
- The AI tools you used and how (see the section above).

**In short:** a hosted, runnable project in a Git repo, with a clear README that explains how it works, why you built it that way, and how AI factored into your process. There is no separate live walkthrough — your repository and README are the deliverable.

## ASSIGNMENT A

# Dynamic Form Builder Engine

Target level: Intermediate

---

## 1. Context & Business Problem

In many modern enterprise applications, user-facing forms, legal disclosures, and input workflows change frequently due to shifting business requirements, regulatory updates, or localisation needs. Hardcoding these forms into traditional static database tables leads to high development overhead, constant redeployments, and complex data migration paths.

To solve this, we want to build a **Dynamic Form Builder Engine**. Instead of mapping forms to static database tables, form fields are represented as dynamic configurations. Your task is to design and implement a flexible prototype that dynamically defines, validates, stores, and renders form responses.

## 2. Core Functional Objectives

Your service must accomplish three main goals:

- **Form Configuration:** store a dynamic form's layout and rules.
- **Dynamic Validation:** accept user form submissions and validate the payload against those rules before persisting it.
- **Submission Storage:** safely store the actual user responses against the form that produced them.

## 3. Recommended Requirements & Architecture

You have full creative control over how you structure your database and application code. The breakdown below describes what the system needs to support.

### A. Data Modelling

You need a storage strategy (e.g. PostgreSQL with JSONB, a document database, or any database of your choice) that handles two primary models:

- **Form Configurations / Templates** — represent the form's fields, types, and validation rules (standard formats such as JSON Schema are highly recommended).
- **Form Submissions** — store the actual user responses safely, linked to the configuration that produced them.

Your backend should expose a clean interface (REST, GraphQL, or RPC) to interact with the engine.

### B. Validation Strategy

To keep the engine truly dynamic, avoid hardcoding field-specific validation rules (like `if (age < 18)`) directly in your application code. Instead, let your form configuration define the validation rules, and use a flexible validation engine or library to parse and run those rules dynamically at runtime.

## C. Frontend

Because this assessment evaluates full-stack ability, include a **minimal frontend** that demonstrates the engine end-to-end:

- Fetch a form configuration and dynamically render its fields (you do not need a drag-and-drop form designer — rendering a stored configuration is enough).
- Let a user fill in and submit the form, surfacing validation errors clearly.
- Handle and display loading, error, and success states.

## 4. Tech Stack Flexibility

You are free to choose the technologies you are most comfortable with, provided they support clean, maintainable, modern code:

- **Language:** Node.js (TypeScript), Go, Python, or similar.
- **Frontend:** any modern framework you are comfortable with (React, Vue, Svelte, etc.).
- **Database:** any database that can comfortably handle dynamic schemas and structured relationships.
- **Setup:** provide a containerised setup (e.g. a simple `docker-compose.yml`) or a straightforward script so we can easily spin up and test your environment locally, in addition to the hosted deployment.

## 5. Trade-off Analysis (in your README)

Rather than a live session, document your thinking in the README. Be ready to explain:

- **Design decisions** — your database schema choice, routing structure, and validation strategy.
- **Implementation details** — your error handling and how your code guarantees data consistency and historical integrity (e.g. how a stored submission stays valid against the configuration version that produced it).
- **Trade-offs** — the architectural trade-offs you made within this short exercise, and how you would scale this engine in a production-ready cloud environment.

## ASSIGNMENT B

# Submission & Approval Workflow

Target level: Intermediate

## 1. Context

Build a small two-sided web app for a generic request submission and approval process. One type of user (an Applicant) submits an application and tracks its status. Another (a Reviewer) sees incoming applications and approves or rejects them with comments.

The heart of this exercise is a **correctly enforced status workflow with an audit trail** — not a large feature set. We care more about clean, well-reasoned, well-tested code on a small surface than about breadth.

## 2. Roles

- **Applicant** — creates, edits (while still a draft), and submits applications; views their own list and statuses.
- **Reviewer** — sees the queue of submitted applications, opens one, and approves / rejects / returns it for changes with a comment.

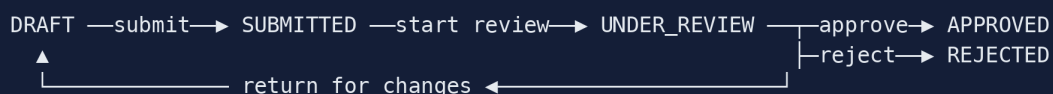
## 3. The Application Object

A simple form is enough — for example:

- title (text, required)
- category (select from a fixed list)
- description (text)
- an amount or date field
- (optional) a single file attachment, if you want to demonstrate file handling.

## 4. The Workflow (the core of the exercise)

A state machine the backend must enforce — illegal transitions are rejected with a clear error:



### Rules:

- Only the owner can edit/submit a DRAFT. Only a Reviewer can move an application out of SUBMITTED / UNDER\_REVIEW.
- An applicant cannot edit an application once it has left DRAFT.
- **reject** and **return for changes** require a comment.
- Every transition is recorded in an audit log (who, old → new status, comment, timestamp) and shown on the detail page.

## 5. Functional Requirements

1. **Auth + roles** — login with at least one Applicant and one Reviewer (seeded users are fine). No need for production-grade auth, but role checks must be real and enforced server-side.
2. **Applicant views** — create/edit draft form (with validation), submit, and a “my applications” list with statuses.
3. **Reviewer views** — a queue list (filterable by status), a detail view, and approve/reject/return actions with a required comment on reject/return.
4. **Backend API** — RESTful endpoints with proper status codes, server-side validation, and authorization enforced on every mutation (an applicant must not be able to approve their own application even by calling the API directly).
5. **Persistence** — a real database; include migrations or a seed script.

## 6. Non-Functional Requirements (what separates intermediate from junior)

- **Tests are required.** At minimum: unit tests covering the state-machine transition rules (both legal and illegal transitions), and at least one API test asserting that an unauthorized action is rejected.
- **Authorization is tested, not assumed** — show that a forbidden transition returns 403.
- **Clear error handling** — validation errors, illegal transitions, and not-found cases all return structured responses.
- **README** — how to run it, your data model, and the trade-offs you made / what you’d add with more time.

## 7. Stack (prescribed)

A standard, neutral web stack so submissions are comparable:

- **Frontend:** React (TypeScript) — any meta-framework or plain Vite/React is fine. State/data-fetching library is your choice.
- **Database:** PostgreSQL, with migrations or a seed script included.
- **Run:** a `docker-compose.yml` (or equivalent documented steps) that brings up the database and backend; frontend run steps documented separately.

Within these, library choices are yours — we want to see you justify anything non-obvious. AI assistants are allowed; you must understand and be able to explain every line you submit (document your AI usage in the README, as described in **Use of AI Tools**).

## 8. What We Are Evaluating (rubric)

| Area                                 | Weight | What “good” looks like                                                                  |
|--------------------------------------|--------|-----------------------------------------------------------------------------------------|
| Workflow correctness & rules         | 25%    | State machine behaves exactly as specified; illegal transitions are blocked.            |
| Backend / API design + authorization | 25%    | Sensible endpoints, status codes, validation; authorization enforced on every mutation. |

|                            |     |                                                                                        |
|----------------------------|-----|----------------------------------------------------------------------------------------|
| Frontend structure & UX    | 20% | Clean components; loading / error / result states all handled; accessible forms.       |
| Testing                    | 15% | Meaningful tests on transition rules plus an authorization test — not just happy-path. |
| Data modeling & migrations | 10% | Clear schema, sensible relationships, reproducible setup.                              |
| Communication              | 5%  | README clarity, documented trade-offs, readable commit history.                        |

*If you run out of time, ship less but ship it solid, and note what you cut.*

## 9. Stretch Goals (optional — pick at most one)

- **Notifications:** email or in-app notification on status change.
- **Queue power-ups:** pagination + search/filter on the reviewer queue.
- **Revision round-trip:** a full “returned for changes” cycle with a visible revision history.

**Do not attempt stretch goals at the expense of tests or a clean core.**

## 10. Suggested 3-Day Plan (guidance, not a mandate)

- **Day 1** — data model, migrations, auth/roles, and the state machine with its unit tests. This is the core; make it solid.
- **Day 2** — the API (CRUD + transition endpoints), authorization, and API tests. End the day able to drive the full workflow via curl / an HTTP client.
- **Day 3** — frontend (applicant + reviewer views), wiring, the three UI states, README, and a final test pass. Only attempt a stretch goal if time remains.

---

*Good luck — we’re looking forward to seeing how you think.*